

Plan de proyecto de software: ciclo de vida, roles, requerimientos y metodología del sistema

¿Quién lo tiene?

Daniel Montenegro Almendra
Brayan Grajales Escarraga
Monica Anyi Cajas Mellizo
Yanimar Burbano Piamba
Wilson Javier Galindez Certuche
Francisco José Castro Muñoz
Deiner Stiven Ramírez Vente

Equipo Byte Force

Fundación Universitaria de Popayán

Facultad de Ingeniería

Ingeniería del Software III

Docente: [Nombre del docente]

29 de octubre de 2026

Tabla de contenido

Introducción 4

Objetivos 5

Ciclo de vida del software 5

Criterios de selección 5

Modelo seleccionado: híbrido Scrum + DevOps 6

Fases del ciclo de vida en el proyecto 7

Roles del equipo 8

Matriz de responsabilidades (RACI) 10

Requerimientos del sistema 10

Estándares aplicados 11

Actores del sistema 11

Requerimientos del negocio 12

Requerimientos funcionales 12

Requerimientos no funcionales 14

Épicas e historias de usuario 15

Validación de los requisitos 16

Metodología y plan de ejecución 16

Metodología: Scrum con prácticas DevOps 16

Plan de sprints 18

Asignación de trabajo por rol 20

Riesgos 20

Herramientas 21

Conclusiones 21

Referencias 22

Introducción

¿Quién lo tiene? es una aplicación web adaptada al celular (aplicación web progresiva, PWA) en la que un cliente publica el producto que busca y las tiendas cercanas responden si lo tienen, a qué precio y si hacen domicilio. El sistema resuelve un problema cotidiano: para conseguir un producto hoy es necesario llamar o visitar varias tiendas, una por una, con pérdida de tiempo y dinero, mientras que las tiendas pequeñas, que no tienen inventario digital, pierden ventas porque los clientes no saben que tienen lo que buscan.

Este documento reúne las decisiones de planificación del equipo Byte Force para construir el producto mínimo viable (MVP) que se entrega el 26 de noviembre de 2026. Presenta el ciclo de vida seleccionado y su justificación, los roles y responsabilidades de los siete integrantes, los requerimientos del sistema organizados según los estándares ISO/IEC/IEEE 29148:2018, IEEE 830-1998 e ISO/IEC 25010, y el plan de ejecución con la metodología Scrum complementada con prácticas DevOps.

El alcance del MVP incluye el registro de clientes, tiendas y administrador; la publicación de solicitudes y su difusión a tiendas en un radio de 3 km; las respuestas y su comparación; las órdenes para recoger o con domicilio; las notificaciones en tiempo real; las reseñas y el panel de administración. Quedan fuera de esta entrega el cobro dentro de la aplicación, el plan de pago para tiendas, la supresión de cuenta y el inicio de sesión sin contraseña.

Objetivos

Objetivo general

Planificar el desarrollo del MVP de ¿Quién lo tiene? definiendo el ciclo de vida, los roles, los requerimientos y la metodología que permitan entregar una aplicación funcional, documentada y probada el 26 de noviembre de 2026.

Objetivos específicos

- Seleccionar y justificar un modelo de ciclo de vida acorde con el tiempo, el tamaño del equipo y los cambios esperados del proyecto.
- Definir los roles y las responsabilidades de cada integrante del equipo y su correspondencia con los roles de Scrum.
- Especificar los requerimientos de negocio, funcionales y no funcionales de acuerdo con los estándares de ingeniería de requisitos y de calidad de producto.
- Establecer el plan de sprints, las ceremonias, los artefactos y los criterios de terminado que guiarán la ejecución.

Ciclo de vida del software

El ciclo de vida del software es el conjunto de etapas por las que pasa un producto desde que se concibe hasta que deja de usarse: análisis de requisitos, diseño, construcción, pruebas, despliegue y mantenimiento (Sommerville, 2011). La norma ISO/IEC/IEEE 12207 organiza estas etapas como procesos del ciclo de vida, y cada modelo (cascada, incremental, espiral o ágil) define en qué orden y con qué frecuencia se recorren (Pressman & Maxim, 2020).

Criterios de selección

Para elegir el modelo se compararon diez alternativas con ocho criterios propios del proyecto: escalabilidad, tiempo de entrega, manejo de la complejidad, gestión de riesgos, tamaño

del equipo, presupuesto, cambios esperados y necesidades del negocio. Cada criterio se calificó de 1 a 5, donde 5 es el mejor ajuste, para un máximo de 40 puntos. Los factores determinantes del contexto son tres: un plazo fijo de unas 4,5 semanas (del 26 de octubre al 26 de noviembre de 2026), un equipo de siete personas y un alcance que todavía puede ajustarse con la rúbrica del docente.

Tabla 1

Matriz comparativa de modelos de ciclo de vida

Modelo	Esc.	Tiem.	Comp.	Ries.	Equipo	Pres.	Camb.	Neg.	Total
Híbrido Scrum + DevOps	5	5	4	5	5	4	5	5	**38**
Scrum	4	4	4	4	5	4	5	5	35
Scrumban	4	4	4	4	4	5	5	4	34
Kanban	4	4	4	3	4	5	5	3	32
Incremental	4	4	4	4	4	4	3	4	31
DevOps (solo)	5	4	3	3	3	4	4	3	29
Evolutivo	3	3	3	3	3	3	4	3	25
Espiral	4	1	2	5	2	2	4	3	23
RUP	4	1	2	3	3	2	3	3	21
Cascada	2	2	3	1	3	4	1	2	18

Nota. Escala de 1 a 5 (5 = mejor ajuste). Esc. = escalabilidad; Tiem. = tiempo de entrega; Comp. = complejidad; Ries. = riesgos; Pres. = presupuesto; Camb. = cambios esperados; Neg. = necesidades del negocio. Total máximo: 40 puntos.

Modelo seleccionado: híbrido Scrum + DevOps

El equipo adopta un modelo híbrido que combina el marco ágil Scrum, con sprints de una semana, y las prácticas DevOps de integración y despliegue continuos. Este modelo obtuvo 38 de 40 puntos, frente a 35 de Scrum puro y 18 de Cascada. Las razones principales son las siguientes:

- **Tiempo de entrega.** Con 4,5 semanas, los sprints semanales dan cuatro puntos de control y una versión funcionando al final de cada semana. Cascada, RUP y Espiral consumen demasiado tiempo en fases de análisis antes de producir software.
- **Riesgos.** El mayor riesgo es no terminar. La revisión semanal permite recortar a tiempo las funciones de menor prioridad, y el despliegue continuo elimina el riesgo de que el sistema solo funcione en el equipo del desarrollador el día de la entrega.
- **Tamaño del equipo.** Siete personas es un tamaño adecuado para un equipo Scrum, que la guía recomienda de diez personas o menos (Schwaber & Sutherland, 2020).
- **Cambios esperados.** El alcance todavía se ajusta. Scrum absorbe los cambios en el Product Backlog sin rehacer el plan, algo que un modelo secuencial no permite.
- **Escalabilidad y presupuesto.** Las prácticas DevOps (pipeline en GitHub Actions, imágenes Docker y despliegue automatizado) son las mismas que servirán cuando el producto crezca, y todas las herramientas son gratuitas; el único costo es el servidor privado virtual (VPS).
- **Necesidades académicas.** Scrum produce directamente los artefactos que exige la ingeniería de requisitos (épicas, historias de usuario y criterios de aceptación) y deja evidencia de cada sprint para la evaluación.

Se descartaron Kanban y Scrumban porque, sin iteraciones de fecha fija, se pierde el ritmo de entregas parciales frente a una fecha límite inamovible. No obstante, dentro de cada sprint el equipo usa un tablero Kanban para visualizar el flujo de trabajo.

Fases del ciclo de vida en el proyecto

Aunque el modelo es iterativo, cada sprint recorre las etapas clásicas del ciclo de vida en pequeño. La Tabla 2 muestra cómo se distribuyen esas etapas en el proyecto.

Tabla 2

Etapas del ciclo de vida aplicadas al proyecto

Etapa	Cómo se realiza en el proyecto	Responsable principal
Análisis de requisitos	Especificación de requisitos (SRS), épicas, historias de usuario y criterios de aceptación en Gherkin; refinamiento del backlog en cada planificación.	Documentador y Product Owner
Diseño	Arquitectura de monolito modular con Clean Architecture, diagramas UML en draw.io y prototipos de interfaz.	Arquitecto de software, UX Designer
Construcción	Desarrollo de la API REST en Java y Spring Boot y del frontend React por historias de usuario, con revisión por pull request.	Arquitecto de software, Developer, UX Designer
Pruebas	Pruebas unitarias, de integración (Testcontainers), de aceptación (Cucumber), de punta a punta (Playwright), de carga (k6) y de seguridad.	QA
Despliegue	Pipeline de CI/CD en GitHub Actions que construye imágenes Docker y despliega en el VPS desde el Sprint 0.	Arquitecto de software
Mantenimiento	Corrección de defectos en cada sprint, copias de seguridad diarias y monitoreo de disponibilidad durante la evaluación.	Equipo completo

Roles del equipo

El equipo Byte Force está formado por siete integrantes. Cada uno tiene un rol de proyecto que define su especialidad y, a la vez, un rol dentro del marco Scrum. Según la guía de Scrum, el equipo tiene tres responsabilidades: el Product Owner, que maximiza el valor del producto y gestiona el Product Backlog; el Scrum Master, que facilita el uso del marco y elimina impedimentos; y los Developers, que construyen el incremento de cada sprint (Schwaber & Sutherland, 2020). En Scrum todos los especialistas (diseño, pruebas y documentación) forman parte de los Developers.

Tabla 3

Integrantes, roles y responsabilidades

Integrante	Rol del proyecto	Rol en Scrum	Responsabilidades principales
Daniel Montenegro Almendra	Arquitecto de Software / Developer	Developer (líder técnico)	Define la arquitectura y los estándares de código; desarrolla los módulos centrales del backend junto con Deiner; crea la estructura inicial del frontend (proyecto React, rutas, cliente de la API y componentes base) para que Monica trabaje sobre un plan definido; configura el repositorio, el pipeline de CI/CD, Docker y el VPS; revisa los pull requests.
Brayan Grajales Escarraga	Líder de Proyecto	Scrum Master	Facilita las ceremonias de Scrum (planning, daily, review y retrospectiva); elimina los impedimentos del equipo; hace seguimiento del avance y los riesgos; vela por el uso correcto de Scrum; es el enlace con el docente.
Monica Anyi Cajas Mellizo	UX Designer	Developer (diseño y frontend)	Diseña los flujos, wireframes y prototipos y define la guía visual; desarrolla todas las pantallas del frontend React (PWA) sobre la estructura inicial e integra la API; vela por la usabilidad y la accesibilidad (WCAG 2.1 AA).
Yanimar Burbano Piamba	QA	Developer (calidad funcional)	Elabora el plan de pruebas; automatiza los escenarios Gherkin con Cucumber y los flujos de punta a punta con Playwright; ejecuta pruebas de regresión y reporta defectos.
Wilson Javier Galindez Certuche	Documentador	Product Owner	Gestiona y prioriza el Product Backlog con MoSCoW; define el objetivo de cada sprint con el equipo; acepta o rechaza las historias según sus criterios de aceptación; mantiene el SRS y la trazabilidad; redacta los manuales y aplica las normas APA a los entregables.
Francisco José Castro Muñoz	QA	Developer (calidad no funcional)	Diseña y ejecuta las pruebas de rendimiento (k6), seguridad (OWASP, Trivy) y autorización por objeto; verifica los requerimientos no funcionales y la cobertura de pruebas.
Deiner Stiven Ramírez Vente	Developer	Developer (backend)	Desarrolla módulos del backend junto con el arquitecto: endpoints REST, persistencia con JPA y PostGIS, migraciones Flyway, notificaciones en tiempo real y mensajería; escribe pruebas unitarias y de integración de su código.

Nota. Siguiendo la guía de Scrum, el Product Owner (Wilson) y el Scrum Master (Brayan) son personas distintas: Wilson decide qué se construye y en qué orden, y Brayan cuida cómo trabaja el equipo. El backend lo desarrollan Daniel y Deiner, y el frontend Monica, a partir de la estructura inicial que deja Daniel en el Sprint 0.

Matriz de responsabilidades (RACI)

La matriz RACI precisa quién realiza cada actividad (R, responsable), quién responde por el resultado (A, aprobador), a quién se consulta (C) y a quién se informa (I).

Tabla 4

Matriz RACI del proyecto

Actividad	Daniel	Brayan	Monica	Yanimar	Wilson	Francisco	Deiner
Gestión del Product Backlog	C	C	C	C	A/R	C	C
Aceptación de historias	I	I	I	C	A/R	C	I
Ceremonias e impedimentos	I	A/R	I	I	C	I	I
Seguimiento de avance y riesgos	C	A/R	I	I	C	I	I
Arquitectura y diseño técnico	A/R	I	C	I	I	C	C
Desarrollo del backend	A/R	I	I	C	I	C	R
Estructura inicial del frontend	A/R	I	C	I	I	I	I
Desarrollo del frontend	C	I	A/R	C	I	I	I
Diseño UX/UI	C	C	A/R	I	I	I	I
Pruebas funcionales	C	I	I	A/R	I	R	C
Pruebas no funcionales	C	I	C	R	I	A/R	I
CI/CD y despliegue	A/R	I	I	C	I	C	R
Documentación y SRS	C	I	C	C	A/R	C	C
Demo y entrega final	R	A/R	R	R	R	R	R

Nota. R = responsable; A = aprobador; C = consultado; I = informado.

Requerimientos del sistema

Los requerimientos se especificaron según la norma ISO/IEC/IEEE 29148:2018, que define los procesos de ingeniería de requisitos y las características de un buen requisito, y con la estructura de la especificación de requisitos de software (SRS) de IEEE 830-1998 (IEEE, 1998; ISO/IEC/IEEE, 2018). Los requerimientos no funcionales se clasificaron con el modelo de calidad de producto de ISO/IEC 25010 (ISO/IEC, 2023). La especificación completa incluye 12 requerimientos de negocio, 42 funcionales, 30 no funcionales, 9 épicas, 38 historias de usuario y 85 casos de prueba.

Estándares aplicados

Tabla 5

Estándares de referencia y su uso en el proyecto

Estándar	Uso en el proyecto
ISO/IEC/IEEE 29148:2018	Proceso de ingeniería de requisitos y validación de cada requisito con ocho criterios: correcto, completo, consistente, no ambiguo, verificable, priorizado, rastreable y factible.
IEEE 830-1998	Estructura del documento SRS: introducción, descripción general y requisitos específicos.
ISO/IEC 25010	Clasificación de los requerimientos no funcionales por característica de calidad (eficiencia de desempeño, seguridad, fiabilidad, mantenibilidad, compatibilidad, usabilidad y portabilidad).
Guía de Scrum (2020)	Épicas, historias de usuario, criterios de aceptación y priorización del backlog.
Ley 1581 de 2012	Protección de datos personales: política de tratamiento y autorización explícita al registrarse.
NIST SP 800-63B	Política de contraseñas: frase de mínimo 15 caracteres y rechazo de contraseñas filtradas.
OWASP ASVS y API Security Top 10	Requisitos de seguridad de la API, en especial la autorización por objeto.
WCAG 2.1	Requisitos de accesibilidad de la interfaz (nivel AA).

Actores del sistema

- **Cliente.** Persona con celular y conocimientos básicos de aplicaciones que publica lo que busca, compara respuestas y hace pedidos.
- **Tienda.** Dueño o vendedor de una farmacia o ferretería, a menudo poco técnico, que responde solicitudes y atiende órdenes.
- **Administrador.** Miembro del equipo que aprueba tiendas, atiende reportes, aplica sanciones y mantiene el catálogo.
- **Sistema.** Procesos automáticos como la difusión por radio, el vencimiento de solicitudes y las notificaciones.

Requerimientos del negocio

Tabla 6

Requerimientos del negocio

ID	Nombre	Descripción	Prioridad
RN-001	Consulta única de disponibilidad	El cliente pregunta una sola vez y la consulta llega a varias tiendas cercanas.	Must
RN-002	Comparación de ofertas	El cliente compara respuestas por precio, distancia, domicilio y calificación.	Must
RN-003	Participación sin inventario digital	La tienda confirma disponibilidad en el momento, sin cargar catálogo.	Must
RN-004	Modalidades de entrega	El cliente recoge en la tienda o pide domicilio, que hace la propia tienda.	Must
RN-005	Tiendas verificadas	Solo aparecen tiendas cuyos documentos aprobó un administrador.	Must
RN-006	Reputación basada en pedidos reales	Solo califica quien resolvió una orden con la tienda.	Must
RN-007	Gestión de incidentes y sanciones	Clientes y tiendas reportan problemas y el administrador puede suspender cuentas.	Must
RN-008	Privacidad del cliente	La tienda no ve los datos personales del cliente hasta ser elegida.	Must
RN-009	Restricción de medicamentos con fórmula	Los medicamentos con fórmula médica solo muestran disponibilidad.	Must
RN-010	Pago contra entrega sin intermediación	La app registra el medio de pago y el total, pero no mueve dinero.	Must
RN-011	Operación de bajo costo	El sistema opera en un solo VPS con herramientas gratuitas.	Must
RN-012	Medición de indicadores	Se registran tiempo de respuesta, tasa de respuesta y calificación.	Should

Requerimientos funcionales

Los 42 requerimientos funcionales se priorizaron con la técnica MoSCoW: 22 Must (imprescindibles), 11 Should (importantes), 5 Could (deseables) y 4 Won't (fuera de esta entrega). Cada requerimiento de la entrega tiene su historia de usuario con el mismo número y sus criterios de aceptación en Gherkin.

Tabla 7

Requerimientos funcionales y su prioridad MoSCoW

ID	Requerimiento	Actor	Prioridad
RF-001	Registrar cliente	Cliente	Must
RF-002	Registrar tienda con documentos	Tienda	Must
RF-003	Iniciar y cerrar sesión	Todos	Must
RF-004	Segundo factor (TOTP) del administrador	Administrador	Should
RF-005	Aceptar política de tratamiento de datos	Cliente, Tienda	Must
RF-006	Verificar correo	Cliente, Tienda	Could
RF-007	Gestionar perfil y direcciones	Cliente	Must
RF-008	Consultar historial y repetir solicitud	Cliente	Could
RF-009	Configurar perfil de tienda	Tienda	Must
RF-010	Configurar domicilio con tarifa fija	Tienda	Must
RF-011	Tarifas de domicilio avanzadas	Tienda	Could
RF-012	Configurar medios de pago	Tienda	Must
RF-013	Abrir o cerrar tienda	Tienda	Must
RF-014	Ver detalle de tienda	Cliente	Should
RF-015	Ver resumen de indicadores	Tienda	Could
RF-016	Publicar solicitud	Cliente	Must
RF-017	Difundir a tiendas elegibles en 3 km	Sistema	Must
RF-018	Ver solicitudes nuevas sin datos personales	Tienda	Must
RF-019	Responder solicitud (Sí, Parecido, No)	Tienda	Must
RF-020	Ver y ordenar respuestas	Cliente	Must
RF-021	Vencer y cerrar solicitudes	Sistema	Must
RF-022	Ampliar radio de 3 a 5 km	Sistema	Could
RF-023	Advertir productos restringidos	Sistema	Should
RF-024	Elegir respuesta y crear orden	Cliente	Must
RF-025	Reservar para recogida	Sistema	Should
RF-026	Ver cómo llegar a la tienda	Cliente	Should
RF-027	Actualizar estado de la orden	Tienda	Must
RF-028	Confirmar recepción	Cliente	Must
RF-029	Cancelar orden	Cliente	Should
RF-030	Notificar en tiempo real (SSE)	Sistema	Must
RF-031	Notificaciones push web	Sistema	Should
RF-032	Contactar por WhatsApp	Cliente, Tienda	Must
RF-033	Mini chat de texto	Cliente, Tienda	Should
RF-034	Calificar tienda	Cliente	Must
RF-035	Reportar problema	Cliente, Tienda	Should
RF-036	Aprobar o rechazar tiendas	Administrador	Must
RF-037	Gestionar reportes y suspender cuentas	Administrador	Should

ID	Requerimiento	Actor	Prioridad
RF-038	Gestionar categorías y restringidos	Administrador	Should
RF-039	Pagar dentro de la app	Cliente	Won't
RF-040	Plan de pago para tiendas	Tienda	Won't
RF-041	Solicitar supresión de cuenta	Cliente, Tienda	Won't
RF-042	Iniciar sesión sin contraseña	Cliente, Tienda	Won't

Nota. La descripción, las precondiciones, las postcondiciones y el criterio de validación de cada requerimiento están en la especificación de requisitos de software (SRS) del proyecto.

Requerimientos no funcionales

Cada requerimiento no funcional tiene una métrica y un criterio de aceptación numérico, de modo que pueda verificarse con pruebas. La categoría corresponde a la característica de calidad de ISO/IEC 25010.

Tabla 8

Requerimientos no funcionales según ISO/IEC 25010

ID	Característica de calidad	Criterio de aceptación
RNF-001	Eficiencia de desempeño	Latencia p95 de la API ≤ 500 ms con 50 usuarios concurrentes.
RNF-002	Eficiencia de desempeño	Una solicitud llega al panel de las tiendas en ≤ 3 s (p95).
RNF-003	Eficiencia de desempeño	Consulta geoespacial ≤ 200 ms con 1.000 tiendas e índice GiST.
RNF-004	Eficiencia de desempeño	API ≤ 640 MB de RAM y stack completo ≤ 2 GB.
RNF-005	Eficiencia de desempeño	Lighthouse móvil ≥ 80 y primera carga ≤ 3 s en 4G.
RNF-006	Seguridad (autenticidad)	Frase de contraseña ≥ 15 caracteres, hash Argon2id o bcrypt.
RNF-007	Seguridad (confidencialidad)	0 accesos indebidos a datos de clientes en pruebas de autorización.
RNF-008	Seguridad (autenticidad)	JWT de 15 min y refresco rotativo en cookie HttpOnly.
RNF-009	Seguridad (resistencia)	Máx. 5 intentos de inicio de sesión/min y 10 solicitudes/hora (HTTP 429).
RNF-010	Seguridad (confidencialidad)	Solo TLS 1.2 y 1.3, HSTS y calificación A en SSL Labs.
RNF-011	Seguridad (integridad)	0 vulnerabilidades críticas o altas sin justificar (Trivy, Dependency-Check).
RNF-012	Seguridad (integridad)	Archivos JPEG, PNG, WebP o PDF ≤ 5 MB, sin metadatos EXIF.
RNF-013	Seguridad (responsabilidad)	100 % de acciones sensibles auditadas, sin datos personales en logs.
RNF-014	Seguridad (confidencialidad)	100 % de usuarios con autorización de datos registrada (Ley 1581).

ID	Característica de calidad	Criterio de aceptación
RNF-015	Fiabilidad (tolerancia a fallos)	0 eventos de dominio perdidos tras reiniciar la API.
RNF-016	Fiabilidad (madurez)	0 transiciones de estado inválidas aceptadas.
RNF-017	Fiabilidad (recuperabilidad)	RPO de 24 h, RTO de 4 h y una restauración probada.
RNF-018	Fiabilidad (disponibilidad)	Disponibilidad ≥ 99 % del 16 al 26 de noviembre de 2026.
RNF-019	Fiabilidad (disponibilidad)	Máximo 60 s fuera de servicio por despliegue.
RNF-020	Flexibilidad (escalabilidad)	100 usuarios concurrentes y 50 conexiones SSE con error < 1 %.
RNF-021	Flexibilidad (escalabilidad)	0 violaciones de fronteras entre módulos (Spring Modulith).
RNF-022	Mantenibilidad (capacidad de prueba)	Cobertura ≥ 70 % en dominio y aplicación (JaCoCo).
RNF-023	Mantenibilidad (modularidad)	0 violaciones de Clean Architecture (ArchUnit).
RNF-024	Mantenibilidad (analizabilidad)	100 % de endpoints en OpenAPI y errores RFC 9457.
RNF-025	Compatibilidad (interoperabilidad)	API REST JSON sobre HTTPS con prefijo /api/v1.
RNF-026	Compatibilidad (coexistencia)	Dos últimas versiones de Chrome, Edge, Firefox y Safari; Android 10+ e iOS 16.4+.
RNF-027	Usabilidad (operabilidad)	Publicar en ≤ 60 s; responder Sí en ≤ 3 toques y No en 1 toque.
RNF-028	Usabilidad (accesibilidad)	Lighthouse ≥ 90, texto ≥ 16 px, botones ≥ 44 px, contraste WCAG AA.
RNF-029	Portabilidad (instalabilidad)	Despliegue en un servidor limpio en ≤ 15 min con Docker Compose.
RNF-030	Portabilidad (adaptabilidad)	0 secretos en el repositorio; configuración por variables de entorno.

Épicas e historias de usuario

Los requerimientos funcionales se agrupan en nueve épicas y 38 historias de usuario que suman 154 puntos de historia (escala Fibonacci): 94 Must, 39 Should y 21 Could. Las historias siguen el formato «Como [rol] quiero [acción] para [beneficio]», por ejemplo: «Como cliente quiero publicar lo que busco para que las tiendas cercanas me digan si lo tienen».

Tabla 9

Épicas del producto

ID	Épica	Historias	Puntos
EP-001	Identidad y acceso	HU-001 a HU-006	21
EP-002	Perfil del cliente	HU-007, HU-008	8
EP-003	Gestión de la tienda	HU-009 a HU-015	23
EP-004	Solicitudes y respuestas	HU-016 a HU-023	39
EP-005	Órdenes y entrega	HU-024 a HU-029	25
EP-006	Comunicación y notificaciones	HU-030 a HU-033	19
EP-007	Reputación	HU-034, HU-035	6
EP-008	Administración	HU-036 a HU-038	13
EP-009	Pagos y monetización (futuro)	Sin historias en esta entrega	0

Validación de los requisitos

Cada requisito se evaluó con los ocho criterios de calidad de ISO/IEC/IEEE 29148. De 84 requisitos (12 de negocio, 42 funcionales y 30 no funcionales), 66 cumplieron todos los criterios en la primera revisión y 18 tuvieron observaciones, que el equipo corrigió o aprobó el 26 de octubre de 2026. Por ejemplo, se fijó en 30 minutos el vencimiento de una solicitud, que antes no tenía valor, y se separaron los flujos de estados de domicilio y de recogida. La trazabilidad es completa: los cinco objetivos de negocio llegan a los requerimientos de negocio, estos a las épicas, las épicas a las historias y las historias a 85 casos de prueba.

Metodología y plan de ejecución

Metodología: Scrum con prácticas DevOps

Scrum es un marco de trabajo ligero que ayuda a las personas y equipos a generar valor mediante soluciones adaptativas para problemas complejos (Schwaber & Sutherland, 2020). Se basa en el empirismo, con tres pilares: transparencia, inspección y adaptación. El equipo lo complementa con prácticas DevOps, que integran el desarrollo y la operación mediante automatización, integración continua y despliegue continuo (Kim et al., 2016).

Ceremonias

Tabla 10

Ceremonias de Scrum en el proyecto

Ceremonia	Frecuencia y duración	Propósito
Sprint Planning	Lunes, 1 hora	Seleccionar las historias del sprint, definir el objetivo y estimar.
Daily Scrum	Diaria, 15 minutos (puede ser asíncrona por chat)	Revisar avance, plan del día e impedimentos.
Sprint Review	Domingo, 30 minutos	Mostrar el incremento desplegado y ajustar el backlog.
Sprint Retrospective	Domingo, 30 minutos	Identificar mejoras del proceso para el siguiente sprint.
Refinamiento del backlog	Continuo durante el sprint	Detallar y reestimar las historias de los próximos sprints.

Artefactos

- **Product Backlog.** Lista priorizada de las 38 historias de usuario, gestionada por el Product Owner en GitHub Projects; su compromiso es el objetivo del producto: el MVP funcional el 26 de noviembre de 2026.
- **Sprint Backlog.** Historias seleccionadas para el sprint y su plan de trabajo; su compromiso es el objetivo del sprint.
- **Incremento.** Versión funcional desplegada en el VPS al final de cada sprint; su compromiso es la definición de terminado.

Definición de terminado

Una historia se considera terminada cuando el código fue revisado por otra persona mediante pull request, las pruebas unitarias y de integración pasan, los criterios de aceptación están verificados, la documentación de la API está actualizada y el cambio está desplegado en el ambiente de pruebas.

Prácticas DevOps

- Repositorio en GitHub con ramas por historia y pull requests que nombran la historia (HU-xxx).
- Integración continua en GitHub Actions: compilación, pruebas, verificación de arquitectura (Spring Modulith y ArchUnit) y escaneo de seguridad en cada cambio.
- Despliegue continuo con imágenes Docker y Docker Compose en el VPS Hostinger KVM 1 desde el Sprint 0.
- Tablero Kanban en GitHub Projects con las columnas Por hacer, En curso, En revisión y Hecho.

Plan de sprints

El proyecto se ejecuta en cuatro sprints de una semana y una semana corta de cierre. El orden sigue la prioridad MoSCoW: primero se construye el camino completo del flujo principal (registrarse, publicar, responder, elegir, entregar y calificar) y después se agregan las funciones Should y Could.

Tabla 11

Plan de sprints

Sprint	Fechas	Objetivo	Historias principales
Sprint 0	26/10 al 01/11	SRS, diagramas UML, roles, repositorio, pipeline de CI/CD, esqueleto Docker desplegado en el VPS y prueba técnica de PostGIS.	Preparación (sin historias de producto)
Sprint 1	02/11 al 08/11	Identidad y roles, registro y aprobación de tiendas, categorías, publicación de solicitudes y difusión por radio.	HU-001 a HU-005, HU-007, HU-009, HU-010, HU-012, HU-013, HU-016, HU-017, HU-036
Sprint 2	09/11 al 15/11	Respuestas Sí, Parecido y No; elección; órdenes y estados; notificaciones en tiempo real.	HU-018 a HU-021, HU-024, HU-027, HU-028, HU-030, HU-032
Sprint 3	16/11 al 22/11	Reseñas, reportes, panel de administración, WhatsApp, mini chat y funciones Should; el VPS queda estable para la evaluación.	HU-034, HU-035, HU-037, HU-038, HU-023, HU-025, HU-026, HU-029, HU-031, HU-033, HU-014
Cierre	23/11 al 26/11	Pruebas finales, correcciones, documentación final y ensayo de la demostración.	Funciones Could si hay capacidad (HU-006, HU-008, HU-011, HU-015, HU-022)

Nota. La asignación de historias a sprints es una propuesta inicial; el equipo la ajusta en cada Sprint Planning según la velocidad real.

Asignación de trabajo por rol

Tabla 12

Trabajo principal de cada integrante por sprint

Integrante	Sprint 0	Sprints 1 a 3	Cierre
Daniel	Repositorio, arquitectura, CI/CD, VPS y estructura inicial del frontend	Módulos de identidad, solicitudes y órdenes; revisión de código; apoyo técnico al frontend	Despliegue final y soporte de la demo
Brayan	Plan de sprints, tablero y calendario de ceremonias	Facilitación de planning, daily, review y retrospectiva; eliminación de impedimentos; control de riesgos	Coordinación de la entrega y la demo
Monica	Flujos, wireframes y guía visual	Pantallas de cliente, tienda y administración en React; integración con la API; pruebas de usabilidad	Ajustes visuales, accesibilidad y corrección de defectos de interfaz
Yanimar	Plan de pruebas	Automatización Cucumber y Playwright por historia	Regresión completa
Wilson	Product Backlog priorizado, SRS y documento de planificación	Refinamiento y priorización del backlog; aceptación de historias; trazabilidad y actas	Aceptación final del incremento; manuales de usuario y de despliegue
Francisco	Herramientas de prueba (k6, Trivy)	Pruebas de seguridad y autorización por historia	Pruebas de carga y de requisitos no funcionales
Deiner	Base de datos PostgreSQL + PostGIS y primeras migraciones	Módulos de tiendas, catálogo, notificaciones, mensajería y reputación; pruebas de su código	Corrección de defectos del backend

Riesgos

Tabla 13

Riesgos del proyecto y su mitigación

ID	Riesgo	Impacto	Mitigación
R-01	El alcance no cabe en 4,5 semanas.	Alto	MoSCoW estricto, sprints de una semana y recorte de Should y Could primero.
R-02	Curva de aprendizaje de PostGIS, SSE y push web.	Medio	Prototipo técnico en el Sprint 0.
R-03	Fuga de datos del cliente hacia tiendas no elegidas.	Alto	Autorización por objeto y DTO por rol, con pruebas específicas.
R-04	Servidor de bajos recursos.	Bajo	Presupuesto de memoria definido en la arquitectura.
R-05	Requisitos del docente no conocidos del todo.	Medio	Solicitar la rúbrica y ajustar el backlog en la siguiente planificación.

Herramientas

- **Gestión:** GitHub Projects (tablero y backlog) y GitHub Issues (una por historia).
- **Desarrollo:** Java y Spring Boot (API REST), React con Vite (PWA), PostgreSQL con PostGIS y Docker Compose.
- **Calidad:** JUnit, Testcontainers, Cucumber, Playwright, k6, ArchUnit, Spring Modulith, JaCoCo, Trivy y OWASP Dependency-Check.
- **Diseño y documentación:** draw.io (diagramas UML), Figma (prototipos) y OpenAPI (documentación de la API).

Conclusiones

El modelo híbrido Scrum + DevOps es el que mejor se ajusta al proyecto: con sprints de una semana y despliegue continuo desde el Sprint 0, el equipo obtiene una versión funcional cada semana y reduce el riesgo de no terminar a tiempo.

La definición de roles asigna a cada integrante una especialidad clara dentro de Scrum, y la matriz RACI evita vacíos y duplicidades en las actividades clave.

La especificación de requerimientos según ISO/IEC/IEEE 29148, IEEE 830 e ISO/IEC 25010 deja requisitos verificables, priorizados y rastreables hasta los casos de prueba, lo que facilita planificar los sprints y demostrar el cumplimiento en la entrega del 26 de noviembre de 2026.

Referencias

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Grassi, P. A., Fenton, J. L., Newton, E. M., Perlner, R. A., Regenscheid, A. R., Burr, W. E., Richer, J. P., Lefkovitz, N. B., Danker, J. M., Choong, Y.-Y., Greene, K. K., & Theofanos, M. F. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63b

Institute of Electrical and Electronics Engineers. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). https://doi.org/10.1109/IEEESTD.1998.88286

International Organization for Standardization & International Electrotechnical Commission. (2023). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): Product quality model* (ISO/IEC 25010:2023).

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2018). *Systems and software engineering: Life cycle processes: Requirements engineering* (ISO/IEC/IEEE 29148:2018). https://doi.org/10.1109/IEEESTD.2018.8559686

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.

OWASP Foundation. (2023). *OWASP API Security Top 10*. https://owasp.org/API-Security/

Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería del software: Un enfoque práctico* (9.ª ed.). McGraw-Hill.

Schwaber, K., & Sutherland, J. (2020). *La guía de Scrum: La guía definitiva de Scrum: Las reglas del juego*. https://scrumguides.org

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/